Project: Unitree G1 Soccer Skills

CS2810 Teaching Team

1 Overview

This is the project component of Embodied AI CS2810 (Spring 2026). The task is to develop reinforcement learning skills for humanoid robot soccer using the Unitree G1 (29 dof) platform in `mjlab` physics simulation. The project centers on two tasks:

- **Shooting**: The G1 robot must approach and kick a stationary ball toward a goal. This is a perception-guided motor skill requiring motion tracking priors and ball trajectory prediction.
- **Goalkeeping**: The G1 robot must intercept an incoming ball launched along a parabolic trajectory. This is a reactive interception task requiring rapid end-effector positioning and ball trajectory estimation.

2 Project Phases & Grading Rubric

The project is evaluated out of 100 total points, broken down into three primary components: Phase 1 (60%), Phase 2 (30% = 15% Shooting + 15% Goalkeeping), and Submission Deliverables (10%).

2.1 Phase 1: Basic Shooting and Goalkeeping (60% of project score)

Implement single-agent training for individual shooter and goalkeeper tasks in a non-adversarial setting. You may refer to [1] and [2] for guidance on environment design, reward shaping, and training strategies. This phase evaluates the fundamental motor control and perception integration of your policies.

The 60% allocation is divided equally between the two tasks (30% each), using a strict tiered accuracy model calculated over 50 evaluation trials:

- **Shooter Policy (30 points total):** Evaluated using `eval_naive_shooter.py`. Success is defined as the ball completely crossing the goal plane inside the frame posts.
- **Goalkeeper Policy (30 points total):** Evaluated using `eval_naive_goalkeeper.py`. Success is defined as a clean block where the ball is intercepted or deflected outside the goal frame.

Grading tiers for both individual components are structured as follows:

Success / Block Rate Brackets	Percentage of Component Credit	Points Awarded per Task
100% Success	100%	30.0 Points
92% – 100% Success	80%	24.0 Points
84% – 92% Success	60%	18.0 Points
80% – 84% Success	50%	15.0 Points
Under 80% Success	0%	0.0 Points

Table 1: Phase 1 Performance Tier Matrix

2.2 Phase 2: Cross-Evaluation Tournament (30% of project score)

Phase 2 is structured as a decentralized, all-to-all peer-to-peer cross-evaluation. The 30% allocation is divided equally between shooting and goalkeeping (15% each). After completing basic single-agent policies in Phase 1, teams continue to improve their agents—through adversarial training, self-play, or further single-agent optimization—and then compete against every other team.

Because each team will design unique observation spaces, feature extractors, and neural network architectures, your codebases will be heterogeneous. Therefore, the tournament relies on teams sharing their repositories and checkpoints so that opponents can load and run them locally.

Important Environment Note: The current template code **does not yet support** loading two distinct G1 robots simultaneously in a single MuJoCo scene. A basic multi-agent competition template is provided as `scripts/compete.py` to help teams get started—it loads two G1 robots into one scene and routes observations to two independent policies. Teams must customize the observation terms in the script to match their own (and their opponent's) policy architectures. See the `CUSTOMIZE_OBSERVATIONS` markers in the script for guidance.

2.2.1 Timeline and Milestones

- **Weeks 13–16: Agent Development & Improvement**
  Teams first complete their basic single-agent shooting and goalkeeping policies (Phase 1). They then continuously improve their agents' capabilities through adversarial training, self-play, or further single-agent optimization. By the end of Week 16 (6.21), every team must freeze their code and publicly publish a GitHub repository containing their complete implementation, training configurations, and final trained checkpoints for both shooter and goalkeeper.
- **Weeks 17–18: Cross-Evaluation & Peer Coordination**
  Teams must contact every other team, clone each other's repositories, and integrate opponent policies into their local multi-agent environment. For each pair of teams (Team A and Team B), both directions are evaluated:
  - Team A's Shooter vs. Team B's Goalkeeper (10 trials)
  - Team B's Shooter vs. Team A's Goalkeeper (10 trials)

  Teams must actively collaborate to cross-verify simulator setups, initial randomizations, and evaluation scripts to ensure consistent and reproducible results.

2.2.2 Scoring Mechanism

There are 14 teams in total. Each team's Phase 2 score is computed separately for shooting and goalkeeping (15 points maximum each):

- **Base Score:** Every team starts with 2 points for each task.
- **Head-to-Head Bonus:** For each opponent team that your agent outperforms, you earn +1 point, up to a maximum of 15 points.

Formally, for a given task (shooting or goalkeeping), let r_i be your agent's success rate against opponent i's counterpart (goals or blocks over 10 trials). Sort all 14 teams in descending order of r_i. A team **outranks** every opponent ranked below it. Your final score for that task is:

$$\text{score} = 2 + \text{number of opponents you outrank}$$

2.2.3 Evaluation Rules and Consistency

1. **Match Execution:** A full cross-evaluation matchup between two teams consists of 10 trials of Team A's Shooter vs. Team B's Goalkeeper, and 10 trials of Team B's Shooter vs. Team A's Goalkeeper.
2. **Mandatory Peer Communication:** Teams must actively collaborate and communicate during Weeks 17–18. Because both teams are executing the cross-evaluations locally, you must cross-verify your simulator setups, initial state randomizations, and evaluation scripts to ensure that the performance metrics recorded by Team A match those recorded by Team B. The results presented in both teams' final reports **must be completely consistent**.

2.3 Submission and Deliverables (10% of project score)

Submission consists of a PDF report and the corresponding code repository before the mid of Week 18 (7.3). The PDF report must include:

- **Phase 1 Results:** Basic shooting and goalkeeping evaluation results (success/block rates over 50 trials) with analysis.
- **Phase 2 Results:** Complete cross-evaluation results against all other teams—your shooter vs. each opponent's goalkeeper, and your goalkeeper vs. each opponent's shooter (10 trials each). Include a summary table of all matchups and the final tournament scores for both tasks.

3 Using the Template

The template is given at https://github.com/xiaojxkevin/G1-mjlab-soccer.

3.1 Repository Structure

The template provides:

- **Playable environments**: `Unitree-G1-Shooter` and `Unitree-G1-Goalkeeper` registered as mjlab tasks with configurable scene layout, ball physics, and MuJoCo visualization.
- **Evaluation scripts**: `eval_naive_shooter.py` and `eval_naive_goalkeeper.py` that load trained checkpoints, run headless batch trials, collect paper-matching metrics, and record videos. And you are allowed to modify these scripts if you want to use different observation spaces, reward functions etc.
- **Training configs**: Training environment designs under `config/training/` that specify observation spaces, reward functions, termination conditions, and domain randomization from the two papers. And it should be noticed that these are generated with coding agents and may contain errors. So you should carefully review and debug them before use.

3.2 Play and Visualization

Use the `scripts/play.py` script to visualize the environment:

```
# Shooter
python scripts/play.py Unitree-G1-Shooter --agent zero --viewer native

# Goalkeeper
python scripts/play.py Unitree-G1-Goalkeeper --agent zero --viewer native
```

3.3 Evaluation

Use the eval scripts to benchmark trained policies:

```
# Shooter
python scripts/eval_naive_shooter.py --headless --num-trials 50 \
    --checkpoint <path>

# Goalkeeper
python scripts/eval_naive_goalkeeper.py --headless --num-trials 50 \
    --checkpoint <path>
```

3.4 Implementing Training

The `train.py` script in `scripts/` serves as the training entrypoint. It is expected to register their own training tasks, implement the model classes, and run PPO training.

3.5 Cross-Evaluation Competition

For Phase 2 cross-evaluation, use the `scripts/compete.py` script:

```
# Headless batch (10 trials, Phase 2 default)
python scripts/compete.py \
    --shooter-checkpoint <team_a_shooter.pt> \
    --goalkeeper-checkpoint <team_b_goalkeeper.pt> \
    --headless --num-trials 10

# Interactive viewer (debugging)
python scripts/compete.py \
    --shooter-checkpoint <path> \
    --goalkeeper-checkpoint <path>
```

The script loads two G1 robots into a single MuJoCo scene, routes observations to two independent policies, concatenates their actions, and reports head-to-head statistics (goals scored, blocks). Teams must customize the observation terms in `make_compete_env_cfg()` to match their policy architectures—see the `CUSTOMIZE_OBSERVATIONS` markers in the script.

3.6 Contributing and Reporting Issues

This template is a work in progress. The training configs, environment wrappers, and evaluation scripts were partially generated by coding agents and may contain bugs or rough edges. **We strongly encourage all students to**:

- **Report issues** — if you find a bug, a broken config, or unclear documentation, open a GitHub Issue on the template repository.
- **Submit pull requests** — fixes, improvements, and additional utilities (e.g., better observation configs, visualization tools, or multi-agent helpers) are welcome. PRs that benefit the whole class will be merged and acknowledged.

Treat this repository as a living codebase that improves with your feedback throughout the semester.

References

[1] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, "Learning soccer skills for humanoid robots: A progressive perception-action framework," 2026. [Online]. Available: https://arxiv.org/abs/2602.05310

[2] J. Ren, J. Long, T. Huang, H. Wang, Z. Wang, F. Jia, W. Zhang, J. Wang, P. Luo, and J. Pang, "Humanoid goalkeeper: Learning from position conditioned task-motion constraints," 2025.